

Object Oriented Programming (Operator Overloading)

Abbas Khalid

April 21, 2025

Table of contents

- 1 Introduction
- 2 Overloading Binary Operators
- 3 Overloading Unary Operators

Operator Overloading

- By default, C++ operators works only with built-in data types like `int` and `float`, and attempting to apply them to objects of a user-defined class will cause complaints from the compiler.
- To use an operator on class objects, an operator must be overloaded. However, there are three exceptions:
 - ① The assignment operator (`=`)
 - ② The address operator (`&`)
 - ③ The comma operator (`,`)
- Even though we can also overload these operators, these may be used with every class object without explicit overloading.
- To overload an operator we only need to write an `operator functions`.

Operator Function

- An operator function defines the operations that the overloaded operator performs relative to the class for which it is created.
- In general, the name of a function that overloads a given operator is composed of the keyword `operator`, followed by the operator that we are overloading i.e.,

```
returnType operator sign (parameters);
```

The sign is the operator to be overloaded.

- In the case of operators that use alphabetic characters, such as `new` and `delete`, we must place at least one space between the keyword `operator` and the operator itself. For other operators, the space is optional.
- Operator functions can be `member functions` or `non-member functions`.

Operator Function

- Overloaded operator functions for binary operators can be **member functions** only when the left operand is an object of the class in which the function is a member.
- When an operator function is implemented as a **member functions**, the leftmost (or only) operand must be an object (or reference to an object) of the operator's class. Member functions use **this** pointer implicitly to obtain one of their class object arguments (the left operand for binary operators). That is why only one operand (right) of a binary operator is explicitly passed during the function call.
- If the left operand is an object of a different class or a built in type, the operator function must be implemented as a **non-member functions**.
- Usually we define a non-member function as a **friend** function. In a non-member function call, parameters for both operands of a binary operator must be explicitly passed.

Operator Function

- We pass the parameter by reference to avoid the unnecessary act of the parameter being copied. A `const` keyword can be used if the member function does not change any of the objects variables.
- The member function returns a reference. This allows us to return the actual object and not a copy of one. This is necessary if we want to use the resulting value in a chain of cascading objects e.g., `a + (b + c)`. First the result of `a+b` will be calculated then this result will be further added to `a` by making another call.

Restrictions on Operator Overloading

- The operator functions for overloading `()`, `[]`, `->` or the assignment `=` Operators must be declared as class members.
- Number of operands cannot be changed. For example, if we are overloading a binary operator, it cannot be used as a unary operator.
- No new operators can be created. Hence, only existing C++ operators are used.
- Overloading can't be performed for the built-in data types. For example, we cannot change how two `ints` are added. That means that operators are overloaded to use with defined data types such as classes.
- Precedence of an operator cannot be changed. For example, the `*` has higher precedence than `+`. This precedence cannot be changed.
- Associativity of an operator cannot be changed. If some operator is right associative, it cannot be changed to be left associative.
- Related operators, like `+` and `+=`, must be overloaded separately.
- Operators `.`, `:`, `::`, `.*`, and `?:` cannot be overloaded.

Overloading Insertion and Extraction Operators

- When compiler see the expression

```
cout<<object;
```

It generates a non-member function call (because leftmost operand is an object of `ostream` class) as follows:

```
operator <<(cout, object);
```

- Similarly,

```
cin>>object;
```

causes a non-member function call (because leftmost operand is an object of `istream` class) as follows:

```
operator >>(cin, object);
```

- We return the `istream` object reference, since this allows us to chain multiple objects to be read in a single line (e.g. `cin >> obj1 >> obj2`). Same is true for `ostream` object (i.e., `cout<<obj1<<obj2`).
- While overloading `<<`, the second parameter is usually declared as `const`.

Example 1

```
1 // Overloading Simple Insertion Operator
2 class Student{
3     public:
4         string name;
5         double cgpa;
6         Student(string n, double gpa) {name=n; cgpa=gpa;}
7 };
8
9 void operator<<(ostream& out, Student& s){
10     out<<"CGPA of student"<<s.name<<"is"<<s.cgpa<<endl;
11 }
12
13 int main(){
14
15     Student s ("Muslim", 3.99);
16     cout<<s;    //Method 1: Implicit call to non-member function operator<<(cout,s)
17     operator<<(cout,s); //Method 2: Explicit call to operator<<(cout,s)
18
19     return 0;
20 }
```

CGPA of student Muslim is 3.99
CGPA of student Muslim is 3.99

Example 2

```
1 // Overloading Cascaded Insertion Operator
2 class Student{
3     public:
4         string name;
5         double cgpa;
6         Student(string n, double gpa) {name=n; cgpa=gpa;}
7 };
8
9 ostream& operator<<(ostream& out, Student& s){
10     out<<"CGPA of student"<<s.name<<"is"<<s.cgpa<<endl;
11     return out;
12 }
13
14 int main(){
15
16     Student s1 ("Muslim", 3.99);
17     Student s2 ("Zakriya", 3.95);
18     cout<<s1<<s2; // Cascaded function calls
19
20     return 0;
21 }
```

CGPA of student Muslim is 3.99
CGPA of student Zakriya is 3.95

Example 3

```
1 // Overloading Cascaded Insertion Operator
2 class Student{
3     string name;
4     double cgpa;
5 public:
6     Student(string n, double gpa) {name=n; cgpa=gpa;}
7     friend ostream& operator<<(ostream&, Student&);
8 };
9
10 ostream& operator<<(ostream& out, Student& s){
11     out<<"CGPA of student"<<s.name<<"is"<<s.cgpa<<endl;
12     return out;
13 }
14
15 int main(){
16     Student s1 ("Muslim", 3.99);
17     Student s2 ("Zakriya", 3.95);
18     cout<<s1<<s2; // Cascaded function calls
19     return 0;
20 }
```

CGPA of student Muslim is 3.99
CGPA of student Zakriya is 3.95

Example 4

```
1  // Overloading Cascaded Insertion and Extraction Operators
2  class Student{
3      string name;
4      double cgpa;
5  public:
6      Student(string n, double gpa) {name=n; cgpa=gpa;}
7      friend ostream& operator<<(ostream&, Student&);
8      friend istream& operator>>(istream&, Student& );
9  };
10
11 ostream& operator<<(ostream& out, Student& s){
12     out<<"CGPA of student"<<s.name<<"is"<<s.cgpa<<endl;
13     return out;
14 }
15
16 istream& operator>>(istream& in, Student& s) {
17     cout<<"Enter Student Name:"; in>>s.name;
18     cout<<"Enter Student CGPA:"; in>>s.cgpa;
19     return in;
20 }
```

Example 4 (Cont.)

```
1
2 int main(){
3
4     Student s1,s2;
5     cin>>s1>>s2;
6     cout<<s1<<s2;
7
8     return 0;
9 }
```

```
Enter Student Name: Muslim
Enter Student CGPA: 3.99
Enter Student Name: Zakriya
Enter Student CGPA: 3.95
CGPA of student Muslim is 3.99
CGPA of student Zakriya is 3.95
```

Arithmetic Operators

- C++ arithmetic operators include the plus operator (+), minus operator (-), multiplication operator (*), division operator (/), and modulus operator (%).
- All of the arithmetic operators are binary operators.
- All of these operators are overloaded in the exact same way.
- Operator functions for these operators can be implemented as member functions as well as non-member functions.
- We can also overload compound operators +=, -=, *=, /= etc. like simple arithmetic operators.

Example 1

```
1 class Fraction {
2     int num, den;
3     public:
4         Fraction(int num = 0, int den = 1)
5             { this->num = num; this->den = den; }
6         Fraction operator+(const Fraction &rhs){
7             Fraction temp;
8             temp.den = this->den * rhs.den;
9             temp.num = rhs.den*this->num+this->den*rhs.num;
10            return temp;
11        }
12        void print(){cout<<num<<"/"<<den<<endl;}
13 };
14 int main() {
15     Fraction a(1,2); Fraction b(1,2); Fraction c;
16     cout<<" a = "; a.print();
17     cout<<" b = "; b.print();
18     c = a + b;
19     cout<<" a + b = "; c.print();
20     return 0;
21 }
```

a = 1/2
b = 1/2
a + b = 4/4

Example 2

```
1 class Fraction {
2     int num, den;
3 public:
4     Fraction(int num = 0, int den = 1)
5         {this->num = num; this->den = den;}
6     Fraction& operator+=(const Fraction &rhs){
7         this->num = rhs.den*this->num+this->den*rhs.num;
8         this->den = this->den*rhs.den;
9         return *this;
10    }
11    void print(){cout<<num<<"/"<<den<<endl;}
12 };
13 int main() {
14     Fraction a(1,2); Fraction c(1,2);
15     cout<<"a ="; a.print();
16     cout<<"c ="; c.print();
17     c += a;
18     cout<<"c += a ="; c.print();
19     return 0;
20 }
```

a = 1/2
c = 1/2
c += a = 4/4

Example 3

```
1 class Fraction {
2     int num, den;
3 public:
4     Fraction(int num = 0, int den = 1)
5         {this->num = num; this->den = den;}
6
7     Fraction operator+(int constant) {
8         Fraction result(num + constant * den, den);
9         return result;
10    }
11    void print(){cout<<num<<"/"<<den<<endl;}
12 };
13
14 int main() {
15     Fraction a(3,2); Fraction b;
16     cout<<"a ="; a.print();
17     b = a + 5;
18     cout<<"b ="; b.print();
19 return 0;
20 }
```

a = 3/2
b = 13/2

Example 4

```
1 class Fraction {
2     int num, den;
3 public:
4     Fraction(int num = 0, int den = 1)
5         {this->num = num; this->den = den;}
6     Fraction operator+(int constant) const { // must be const
7         Fraction result(num + constant * den, den);
8         return result;
9     }
10    friend Fraction operator+(int c, const Fraction& f) {
11        return (f + c); // Reuse the member operator. N.B. f is const
12    }
13    void print(){cout<<num<<"/"<<den<<endl;}
14 };
15 int main() {
16     Fraction a(3,2); Fraction b;
17     cout<<"a ="; a.print();
18     b = 5 + a;
19     cout<<"b ="; b.print();
20     return 0;
21 }
```

a = 3/2
b = 13/2

Relational and Logical Operators

- We know, an overloaded operator function usually returns an object of the class for which it is overloaded. However, an overloaded relational and logical operator typically returns a `true` or `false` value.
- All of the relational and logical operators are binary operators.
- Operator functions for these operators can be implemented as member functions as well as non-member functions.
- Overloading operator `&&` or `||` is generally not recommended. They are short-circuited, i.e. don't evaluate their right-hand argument if not necessary. We can't get this behaviour by overloading, since we will be defining a function.
- It would probably be a better choice to overload `&` and `|`.

Example

```
1 class Fraction {
2     int num, den;
3 public:
4     Fraction(int num = 0, int den = 1)
5         {this->num = num; this->den = den;}
6     bool operator!=(const Fraction &rhs){
7         if(this->num/rhs.num != this->den/rhs.den)
8             return true;
9         return false;
10    }
11    bool operator>(const Fraction &rhs){
12        if(this->num/rhs.num > this->den/rhs.den)
13            return true;
14        return false;
15    }
16    void print(){cout<<num<<"/"<<den<<endl;}
17 };
```

Example (Cont.)

```
1 int main() {  
2  
3     Fraction a(1,2); Fraction c(1,2);  
4  
5     cout<<"a="; a.print();  
6     cout<<"c="; c.print();  
7  
8     cout<<"a!=c :"; cout<<(a!=c?"true":"false")<<endl;  
9     cout<<"a>c :";   cout<<(a>c?"true":"false")<<endl;  
10  
11 return 0;  
12 }
```

```
a = 1/2  
c = 1/2  
a != c : false  
a > c : false
```

Optional Example 1

```
1  // Insertion operator as a member function.  Awkward use of operator overloading
2
3  class Student{
4      string name;
5      double cgpa;
6  public:
7      Student(string n, double gpa) {name=n; cgpa=gpa;}
8      ostream& operator<<(ostream& out) {
9          out<<"CGPA of student"<<name<<"is"<<cgpa<<endl;
10         return out;
11     }
12 };
13 int main(){
14
15     Student s1 ("Muslim", 3.99);
16     Student s2 ("Zakriya", 3.95);
17     s2<<(s1<<cout);    // Calls - operator<<(cout), () are necessary.
18
19     return 0;
20 }
```

CGPA of student Muslim is 3.99
CGPA of student Zakriya is 3.95

Optional Example 2

```
1  // + operator using non-member operator function
2  class Frac{
3      int num, den;
4  public:
5      Frac(int n = 0,int d = 1){num = n; den = d;}
6      friend Frac operator+(const Frac&, const Frac&);
7      void print(){cout<<num<<"/"<<den<<endl;}
8  };
9      Frac operator+(const Frac &lhs, const Frac &rhs){
10         Frac temp;
11         temp.den = lhs.den*rhs.den;
12         temp.num = rhs.den*lhs.num + lhs.den*rhs.num;
13         return temp;
14     }
15 int main() {
16     Frac a(1,2); Frac b(1,4); Frac c;
17     cout<<"a="; a.print();
18     cout<<"b="; b.print();
19     c = a + b;
20     cout<<"a + b="; c.print();
21     return 0;
22 }
```

a = 1/2
b = 1/4
a + b = 6/8

Optional Example 3

```
1  // += operator using non-member operator function
2  class Frac{
3      int num, den;
4  public:
5      Frac(int n = 0,int d = 1){num = n; den = d;}
6      friend void operator+=(const Frac &L,const Frac &R){
7          L.num = R.den*L.num + L.den*R.num;
8          L.den = L.den * R.den;
9      }
10     void print(){cout<<num<<"/"<<den<<endl;}
11 };
12
13 int main() {
14     Frac a(1,2); Frac b(1,2);
15     cout<<"a="; a.print();
16     cout<<"b="; b.print();
17     a += b;
18     cout<<"a += b"; a.print();
19     return 0;
20 }
```

a = 1/2
b = 1/2
a += b = 4/4

Optional Example 4

```
1  // Relational operators using non-member operator function
2  class Frac{
3      int num, den;
4  public:
5      Frac(int n = 0,int d = 1){num = n; den = d;}
6      friend bool operator==(const Frac &L, const Frac &R){
7          if(L.num/R.num == L.den/R.den)
8              return true;
9          return false;
10     }
11     friend bool operator<(const Frac &L, const Frac &R){
12         if(L.num/R.num < L.den/R.den)
13             return true;
14         return false;
15     }
16     void print(){cout<<num<<"/"<<den<<endl;}
17 };
```

Optional Example 4 (Cont.)

```
1
2 int main() {
3
4     Fraction a(1,2); Fraction c(1,2);
5
6     cout<<"a="; a.print();
7     cout<<"c="; c.print();
8
9     cout<<"a==c :"; cout<<(a==c?"true":"false")<<endl;
10    cout<<"a<c :";   cout<<(a<c?"true":"false")<<endl;
11
12    return 0;
13 }
```

```
a = 1/2
c = 1/2
a == c : true
a < c : false
```

Pre-increment and Post-increment Operators

- We may also overload unary operators, such as ++, --, or the unary - or +.
- When a unary operator is overloaded by means of a member function, no object is explicitly passed to the operator function. Instead, the operation is performed on the object that generates the call to the function through the implicitly passed `this` pointer.
- When a `friend` is used to overload a unary operator, a single operand is passed.
- As we know, the ++ and -- have both a prefix and a postfix form. In order to distinguish the postfix implementation from the prefix implementation, a dummy parameter `int` is passed. When C++ sees this parameter in an operator function, it knows the function is designed to be used in postfix mode. There is no other use of this dummy parameter.

Example 1

```
1 class Fraction {
2     int num, den;
3 public:
4     Fraction(int num = 0, int den = 1)
5         {this->num = num; this->den = den;}
6
7     Fraction& operator++(){
8         num += den;
9         return *this;
10    }
11
12    Fraction operator++(int){ // Parmater's name is optional
13        Fraction temp(num, den);
14        num += den;
15        return temp;
16    }
17
18    void print(){cout<<num<<"/"<<den<<endl;}
19 };
```

Example 1 (Cont.)

```
1  int main() {  
2  
3      Fraction a(1,2); Fraction c;  
4      cout<<" a = "; a.print();  
5  
6      c = ++a;  
7  
8      cout<<" ++a = "; c.print();  
9      cout<<" a = "; a.print();  
10  
11     c = a++;  
12  
13     cout<<" a++ = "; c.print();  
14     cout<<" a = "; a.print();  
15  
16     return 0;  
17 }
```

a = 1/2
++a = 3/2
a = 3/2
a++ = 3/2
a = 5/2

Example 2

```
1 class Fraction {
2     int num, den;
3 public:
4     Fraction(int num = 0, int den = 1)
5     {this->num = num; this->den = den;}
6
7     Fraction operator-()
8     {Fraction temp(-num,den); return temp;}
9
10    void print(){cout<<num<<"/"<<den<<endl;}
11 };
12
13 int main() {
14
15     Fraction a(1,2); Fraction c;
16     cout<<" a = "; a.print();
17     c = -a;
18     cout<<" -a = "; c.print();
19
20     return 0;
21 }
```

a = 1/2
-a = -1/2

Optional Example 1

```
1 class Fraction {
2     int num, den;
3 public:
4     Fraction(int num = 0, int den = 1)
5     {this->num = num; this->den = den;}
6
7     friend Fraction operator-(const Fraction& op)
8     {Fraction temp(-op.num,op.den); return temp;}
9
10    void print(){cout<<num<<"/"<<den<<endl;}
11 };
12
13 int main() {
14
15     Fraction a(1,2); Fraction c;
16     cout<<" a = "; a.print();
17     c = -a;
18     cout<<" -a = "; c.print();
19
20     return 0;
21 }
```

a = 1/2
-a = -1/2

Optional Example 2

```
1  class Fraction {
2      int num, den;
3  public:
4      Fraction(int num = 0, int den = 1)
5          {this->num = num; this->den = den;}
6
7      friend Fraction operator++(Fraction &op){
8          op.num += op.den;
9          Fraction temp(op.num, op.den);
10         return temp;
11     }
12
13     friend Fraction operator++(Fraction &op, int){
14         Fraction temp(op.num, op.den);
15         op.num += op.den;
16         return temp;
17     }
18     void print(){cout<<num<<"/"<<den<<endl;}
19 };
```


Optional Example 2 (Cont.)

```
1  int main() {  
2  
3      Fraction a(1,2); Fraction c;  
4      cout<<" a = "; a.print();  
5  
6      c = ++a;  
7  
8      cout<<" ++a = "; c.print();  
9      cout<<" a = "; a.print();  
10  
11     c = a++;  
12  
13     cout<<" a++ = "; c.print();  
14     cout<<" a = "; a.print();  
15  
16     return 0;  
17 }
```

a = 1/2
++a = 3/2
a = 3/2
a++ = 3/2
a = 5/2

Conclusions

In this lecture we discussed about

- Operator Overloading
- Operator Functions
- Overloading Binary Operators
- Overloading Unary Operators

In the next lecture we will discuss about

- Association
- Aggregation
- Composition